

HandyServe Pro

Database Schema & SQL Queries

Oracle Database · MongoDB Collections

Contents

- 1. Database Overview
- 2. Oracle Sequences
- 3. Oracle Table Schemas
 - 3.1 HS_USERS
 - 3.2 HS_BOOKINGS
 - 3.3 HS_PROMO_CODES
 - 3.4 HS_LEAVE_REQUESTS
- 4. MongoDB Collection Schemas
 - 4.1 disputes
 - 4.2 notifications
 - 4.3 contact_requests
- 5. Complete SQL Scripts
 - 5.1 Drop Script
 - 5.2 Sequence Creation
 - 5.3 Table Creation
 - 5.4 Indexes
 - 5.5 Additional Constraints
- 6. Key Queries

1. Database Overview

HandyServe Pro uses a dual-database architecture — Oracle for structured relational data and MongoDB for flexible document data.

Database	Tables / Collections	Role
Oracle (JPA)	HS_USERS, HS_BOOKINGS, HS_PROMO_CODES, HS_LEAVE_REQUESTS	ACID transactions, referential integrity, relational joins
MongoDB	disputes, notifications, contact_requests	Schema-flexible documents, nested arrays, high-volume writes

Cross-database references — MongoDB documents reference Oracle IDs as plain Long fields (soft references; enforced at the application layer, not by DB constraints).

2. Oracle Sequences

Oracle does not have AUTO_INCREMENT. Dedicated sequences generate primary keys. NOCACHE prevents ID gaps after restarts.

Sequence	Used By	Start / Increment
HS_USER_SEQ	HS_USERS	1 / 1
HS_BOOKING_SEQ	HS_BOOKINGS	1 / 1
HS_LEAVE_SEQ	HS_LEAVE_REQUESTS	1 / 1
HS_PROMO_SEQ	HS_PROMO_CODES	1 / 1

3. Oracle Table Schemas

3.1 HS_USERS

Central user table for all three roles — Customer, Service Provider, and Admin.

Sequence: HS_USER_SEQ

Column	Type / Size	Nullable	Default	Constraint	Description
id	NUMBER(19, 0)	NOT NULL	HS_USER_SEQ	PRIMARY KEY	System-generated unique user ID

Column	Type / Size	Nullable	Default	Constraint	Description
name	VARCHAR2 (100)	NOT NULL	—	—	Full display name
email	VARCHAR2 (150)	NOT NULL	—	UNIQUE	Login email – unique across all users
phone	VARCHAR2 (20)	NULL	—	—	Mobile number (optional)
password	VARCHAR2 (255)	NOT NULL	—	—	BCrypt-hashed password
role	VARCHAR2 (20)	NOT NULL	—	IN('customer', 'provider', 'admin')	Role discriminator
avatar	VARCHAR2 (10)	NULL	—	—	Two-letter initials (e.g. RK)
verified	NUMBER(1,0)	NOT NULL	0	IN(0,1)	Email verification flag
blocked	NUMBER(1,0)	NOT NULL	0	IN(0,1)	Admin block flag
state	VARCHAR2 (100)	NULL	—	—	State / Province
city	VARCHAR2 (100)	NULL	—	—	Home city
service_city	VARCHAR2 (100)	NULL	—	—	City where provider offers services
service_city_active	NUMBER(1,0)	NOT NULL	1	IN(0,1)	Service city active flag
location	VARCHAR2 (200)	NULL	—	—	Locality / neighbourhood name
display_address	VARCHAR2 (500)	NULL	—	—	Full human-readable address
address	VARCHAR2 (300)	NULL	—	—	Structured delivery address
pincode	VARCHAR2 (10)	NULL	—	—	Postal / ZIP code
latitude	NUMBER(12,8)	NULL	—	—	GPS latitude (WGS84)
longitude	NUMBER(12,8)	NULL	—	—	GPS longitude (WGS84)
detected_city_label	VARCHAR2 (100)	NULL	—	—	City auto-detected from GPS
service_type	VARCHAR2 (100)	NULL	—	—	Provider skill (e.g. Plumbing)
experience	VARCHAR2 (50)	NULL	—	—	Years of experience
timing	VARCHAR2 (50)	NULL	—	—	Working hours string
radius	NUMBER(10,0)	NULL	—	—	Service radius in km

Column	Type / Size	Nullable	Default	Constraint	Description
pricing	VARCHAR2 (50)	NULL	—	—	Pricing label (e.g. ₹350)
reliability_score	NUMBER(10,0)	NULL	90	BETWEEN 0 AND 100	Provider reliability metric
low_score_days	NUMBER(10,0)	NULL	0	—	Consecutive days below threshold
available	NUMBER(1,0)	NULL	1	IN(0,1)	Live availability toggle
id_type	VARCHAR2 (50)	NULL	—	—	Govt ID type (Aadhaar / PAN)
id_number	VARCHAR2 (50)	NULL	—	—	Masked govt ID number
upi	VARCHAR2 (100)	NULL	—	—	Provider UPI ID
bank_name	VARCHAR2 (100)	NULL	—	—	Bank name for payouts
account_holder	VARCHAR2 (100)	NULL	—	—	Bank account holder name
gender	VARCHAR2 (20)	NULL	—	—	Gender identity
age	NUMBER(10,0)	NULL	—	> 0 AND < 120	Age in years
refresh_token	VARCHAR2 (512)	NULL	—	—	Hashed JWT refresh token
created_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	—	Record creation timestamp
updated_at	TIMESTAMP	NULL	—	—	Last update timestamp

3.2 HS_BOOKINGS

Core transactional table. Links a customer to a provider for a service booking. Includes geolocation snapshots, lifecycle status, and payment details.

Sequence: HS_BOOKING_SEQ

Column	Type / Size	Nullable	Default	Constraint	Description
id	NUMBER(19,0)	NOT NULL	HS_BOOKING_SEQ	PRIMARY KEY	Unique booking ID
service	VARCHAR2 (100)	NOT NULL	—	—	Service category (e.g. Plumbing)
status	VARCHAR2 (30)	NOT NULL	'Requested'	CHECK status	Booking lifecycle state
booking_date	VARCHAR2 (20)	NOT NULL	—	—	Scheduled date YYYY-MM-DD

Column	Type / Size	Nullable	Default	Constraint	Description
booking_time	VARCHAR2 (20)	NULL	—	—	Scheduled time string
amount	NUMBER(10,2)	NOT NULL	0.0	>= 0	Service amount in INR
options	VARCHAR2 (500)	NULL	—	—	Selected sub-options (CSV/JSON)
rating	NUMBER(5,0)	NULL	—	BETWEEN 1 AND 5	Customer star rating
invoice_id	VARCHAR2 (50)	NULL	—	—	Auto-generated on Completed
customer_id	NUMBER(19,0)	NOT NULL	—	FK → HS_USERS.id ON DELETE CASCADE	Booking customer
customer_name	VARCHAR2 (100)	NULL	—	—	Snapshot of customer name
customer_city	VARCHAR2 (100)	NULL	—	—	Snapshot of customer city
customer_lat	NUMBER(12,8)	NULL	—	—	Customer GPS latitude
customer_lon	NUMBER(12,8)	NULL	—	—	Customer GPS longitude
customer_address	VARCHAR2 (500)	NULL	—	—	Full service delivery address
customer_directions_url	VARCHAR2 (500)	NULL	—	—	Google Maps URL for provider
nav_to_customer_url	VARCHAR2 (500)	NULL	—	—	Deep-link navigation URL
provider_id	NUMBER(19,0)	NOT NULL	—	FK → HS_USERS.id ON DELETE CASCADE	Assigned provider
provider_name	VARCHAR2 (100)	NULL	—	—	Snapshot of provider name
provider_city	VARCHAR2 (100)	NULL	—	—	Snapshot of provider city
provider_lat	NUMBER(12,8)	NULL	—	—	Provider GPS latitude
provider_lon	NUMBER(12,8)	NULL	—	—	Provider GPS longitude
payment_method	VARCHAR2 (50)	NULL	—	—	UPI / Card / Cash
payment_id	VARCHAR2 (100)	NULL	—	—	Razorpay transaction ID
paid_at	TIMESTAMP	NULL	—	—	Payment verification timestamp

Column	Type / Size	Nullable	Default	Constraint	Description
created_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	—	Booking creation timestamp
updated_at	TIMESTAMP	NULL	—	—	Last status update timestamp

Status allowed values: Requested · Accepted · In_Progress · Pending_Payment · Completed · Cancelled · Rejected

3.3 HS_PROMO_CODES

Promotional discount codes applied at checkout. Supports percentage and flat-rate discounts with optional expiry.

Sequence: HS_PROMO_SEQ

Column	Type / Size	Nullable	Default	Constraint	Description
id	NUMBER(19,0)	NOT NULL	HS_PROMO_SEQ	PRIMARY KEY	Unique promo ID
code	VARCHAR2(30)	NOT NULL	—	UNIQUE	Promo code string (e.g. FIRST50)
label	VARCHAR2(200)	NULL	—	—	Human-readable description
type	VARCHAR2(20)	NOT NULL	—	IN('percent', 'flat')	Discount type
value	NUMBER(10,2)	NOT NULL	—	—	Discount amount or percentage
active	NUMBER(1,0)	NOT NULL	1	IN(0,1)	1=Active, 0=Expired
expires_at	TIMESTAMP	NULL	—	—	Expiry timestamp; NULL = no expiry

3.4 HS_LEAVE_REQUESTS

Providers submit leave/unavailability requests for specific dates and time slots. Admins approve or reject them, preventing booking conflicts.

Sequence: HS_LEAVE_SEQ

Column	Type / Size	Nullable	Default	Constraint	Description
id	NUMBER(19,0)	NOT NULL	HS_LEAVE_SEQ	PRIMARY KEY	Unique leave request ID
provider_id	NUMBER(19,0)	NOT NULL	—	FK → HS_USERS.id ON DELETE CASCADE	Requesting provider

Column	Type / Size	Nullable	Default	Constraint	Description
provider_name	VARCHAR2 (100)	NULL	–	–	Snapshot of provider name
skill	VARCHAR2 (100)	NULL	–	–	Provider service type
leave_date	VARCHAR2 (20)	NOT NULL	–	–	Leave date YYYY-MM-DD
time_slots	VARCHAR2 (500)	NULL	–	–	Comma-separated slots; NULL = full day
reason	VARCHAR2 (1000)	NOT NULL	–	–	Provider's reason for leave
status	VARCHAR2 (20)	NOT NULL	'pending'	IN('pending','approved','rejected')	Admin decision
created_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	–	Submission timestamp

4. MongoDB Collection Schemas

4.1 disputes

Dispute cases raised by customers. The embedded updates[] array stores the full threaded timeline without joins.

Field	Type	Required	Default	Description
_id	ObjectId (String)	YES	Auto	MongoDB document ID
booking_id	Long	YES	–	Soft ref → HS_BOOKINGS.id
customer_id	Long	YES	–	Soft ref → HS_USERS.id (customer)
customer	String	NO	–	Customer display name (denormalised)
customer_email	String	NO	–	Customer email (denormalised)
service_provider_id	Long	YES	–	Soft ref → HS_USERS.id (provider)
provider	String	NO	–	Provider display name (denormalised)
provider_email	String	NO	–	Provider email (denormalised)
issue	String	YES	–	Customer's problem description
issue_category	String	NO	'General issue'	Category: General / Payment / Quality
priority	String	NO	'Medium'	Low / Medium / High
source	String	NO	'web'	Submission source: web / app
amount	Double	NO	–	Disputed amount in INR
status	String	NO	'Open'	Open / Pending / Refund Approved / Resolved / Rejected
date	String	NO	–	Human-readable creation date
updates	Array	NO	[]	Embedded timeline (see sub-fields below)
updates[].id	String	NO	–	Unique entry ID within the array

Field	Type	Required	Default	Description
updates[].actor	String	NO	–	Name of person who added the update
updates[].actor_role	String	NO	–	admin / customer / provider
updates[].note	String	NO	–	Update text / comment
updates[].at	String	NO	–	Human-readable timestamp
created_at	Date	YES	Now	Document creation timestamp

Indexes:

```

db.disputes.createIndex({ booking_id: 1 })
db.disputes.createIndex({ customer_id: 1 })
db.disputes.createIndex({ service_provider_id: 1 })
db.disputes.createIndex({ status: 1 })
db.disputes.createIndex({ created_at: -1 })

```

4.2 notifications

Real-time push notifications persisted for inbox retrieval.

Field	Type	Required	Default	Description
_id	ObjectId (String)	YES	Auto	MongoDB document ID
user_id	Long	YES	–	Soft ref → HS_USERS.id (indexed)
title	String	YES	–	Short notification title
message	String	NO	''	Full notification body
icon	String	NO	'■'	Emoji icon for visual categorisation
read	Boolean	NO	false	false = Unread · true = Read
link	String	NO	''	Frontend deep-link route
created_at	Date	YES	Now	Creation timestamp (indexed)

Indexes:

```

db.notifications.createIndex({ user_id: 1 })
db.notifications.createIndex({ created_at: -1 })
// Compound – efficient 'get unread for user' queries
db.notifications.createIndex({ user_id: 1, read: 1, created_at: -1 })

```

4.3 contact_requests

Public contact / support form submissions. No user account required.

Field	Type	Required	Default	Description
_id	ObjectId (String)	YES	Auto	MongoDB document ID
name	String	YES	–	Submitter's full name
email	String	YES	–	Submitter's email address
phone	String	NO	''	Submitter's phone (optional)
message	String	YES	–	Support inquiry text
status	String	NO	'pending'	pending / replied / closed
date	String	NO	–	Human-readable submission date
created_at	Date	YES	Now	Submission timestamp

5. Complete SQL Scripts

5.1 Drop Script (Safe Reset)

Run only when resetting the schema. Drop child tables before parent tables.

```
-- Sequences
DROP SEQUENCE HS_USER_SEQ;
DROP SEQUENCE HS_BOOKING_SEQ;
DROP SEQUENCE HS_LEAVE_SEQ;
DROP SEQUENCE HS_PROMO_SEQ;
-- Tables (child → parent order)
DROP TABLE HS_LEAVE_REQUESTS CASCADE CONSTRAINTS;
DROP TABLE HS_BOOKINGS CASCADE CONSTRAINTS;
DROP TABLE HS_PROMO_CODES CASCADE CONSTRAINTS;
DROP TABLE HS_USERS CASCADE CONSTRAINTS;
```

5.2 Sequence Creation

```
CREATE SEQUENCE HS_USER_SEQ START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE HS_BOOKING_SEQ START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE HS_LEAVE_SEQ START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE HS_PROMO_SEQ START WITH 1 INCREMENT BY 1 NOCACHE;
```

5.3 Table Creation

HS_USERS

```
CREATE TABLE HS_USERS (
  -- Core Identity
  id NUMBER(19,0) NOT NULL,
  name VARCHAR2(100) NOT NULL,
  email VARCHAR2(150) NOT NULL,
  phone VARCHAR2(20),
  password VARCHAR2(255) NOT NULL,
  role VARCHAR2(20) NOT NULL,
  avatar VARCHAR2(10),
  verified NUMBER(1,0) DEFAULT 0 NOT NULL,
  blocked NUMBER(1,0) DEFAULT 0 NOT NULL,
  -- Location
  state VARCHAR2(100),
  city VARCHAR2(100),
  service_city VARCHAR2(100),
  service_city_active NUMBER(1,0) DEFAULT 1 NOT NULL,
  location VARCHAR2(200),
  display_address VARCHAR2(500),
  address VARCHAR2(300),
  pincode VARCHAR2(10),
  latitude NUMBER(12,8),
  longitude NUMBER(12,8),
  detected_city_label VARCHAR2(100),
  -- Provider-Only
  service_type VARCHAR2(100),
```

```

experience VARCHAR2(50),
timing VARCHAR2(50),
radius NUMBER(10,0),
pricing VARCHAR2(50),
reliability_score NUMBER(10,0) DEFAULT 90,
low_score_days NUMBER(10,0) DEFAULT 0,
available NUMBER(1,0) DEFAULT 1,
-- Verification & Banking
id_type VARCHAR2(50),
id_number VARCHAR2(50),
upi VARCHAR2(100),
bank_name VARCHAR2(100),
account_holder VARCHAR2(100),
gender VARCHAR2(20),
age NUMBER(10,0),
-- Session & Audit
refresh_token VARCHAR2(512),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
updated_at TIMESTAMP,
CONSTRAINT pk_hs_users PRIMARY KEY (id),
CONSTRAINT uq_hs_users_email UNIQUE (email),
CONSTRAINT chk_hs_user_role CHECK (role IN ('customer', 'provider', 'admin')),
CONSTRAINT chk_hs_user_verified CHECK (verified IN (0,1)),
CONSTRAINT chk_hs_user_blocked CHECK (blocked IN (0,1)),
CONSTRAINT chk_hs_user_svc_active CHECK (service_city_active IN (0,1)),
CONSTRAINT chk_hs_user_available CHECK (available IN (0,1))
);
COMMENT ON COLUMN HS_USERS.password IS 'BCrypt-hashed – never stored in plain text.';
COMMENT ON COLUMN HS_USERS.role IS 'Discriminator: customer | provider | admin.';
COMMENT ON COLUMN HS_USERS.reliability_score IS 'Provider metric 0-100. Default 90. Degrades on missed bookings.';
COMMENT ON COLUMN HS_USERS.refresh_token IS 'Hashed JWT refresh token (HttpOnly cookie pairing).';

```

HS_PROMO_CODES (no FK dependencies — create before HS_BOOKINGS)

```

CREATE TABLE HS_PROMO_CODES (
id NUMBER(19,0) NOT NULL,
code VARCHAR2(30) NOT NULL,
label VARCHAR2(200),
type VARCHAR2(20) NOT NULL,
value NUMBER(10,2) NOT NULL,
active NUMBER(1,0) DEFAULT 1 NOT NULL,
expires_at TIMESTAMP,
CONSTRAINT pk_hs_promo_codes PRIMARY KEY (id),
CONSTRAINT uq_hs_promo_codes_code UNIQUE (code),
CONSTRAINT chk_hs_promo_type CHECK (type IN ('percent', 'flat')),
CONSTRAINT chk_hs_promo_active CHECK (active IN (0,1))
);

```

HS_BOOKINGS (depends on HS_USERS)

```

CREATE TABLE HS_BOOKINGS (
id NUMBER(19,0) NOT NULL,
service VARCHAR2(100) NOT NULL,
status VARCHAR2(30) DEFAULT 'Requested' NOT NULL,
booking_date VARCHAR2(20) NOT NULL,
booking_time VARCHAR2(20),
amount NUMBER(10,2) DEFAULT 0.0 NOT NULL,
options VARCHAR2(500),
rating NUMBER(5,0),

```

```

invoice_id VARCHAR2(50),
-- Customer snapshot
customer_id NUMBER(19,0) NOT NULL,
customer_name VARCHAR2(100),
customer_city VARCHAR2(100),
customer_lat NUMBER(12,8),
customer_lon NUMBER(12,8),
customer_address VARCHAR2(500),
customer_directions_url VARCHAR2(500),
nav_to_customer_url VARCHAR2(500),
-- Provider snapshot
provider_id NUMBER(19,0) NOT NULL,
provider_name VARCHAR2(100),
provider_city VARCHAR2(100),
provider_lat NUMBER(12,8),
provider_lon NUMBER(12,8),
-- Payment
payment_method VARCHAR2(50),
payment_id VARCHAR2(100),
paid_at TIMESTAMP,
-- Audit
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
updated_at TIMESTAMP,
CONSTRAINT pk_hs_bookings PRIMARY KEY (id),
CONSTRAINT fk_hs_bookings_cust FOREIGN KEY (customer_id)
REFERENCES HS_USERS (id) ON DELETE CASCADE,
CONSTRAINT fk_hs_bookings_prov FOREIGN KEY (provider_id)
REFERENCES HS_USERS (id) ON DELETE CASCADE,
CONSTRAINT chk_hs_booking_status CHECK (status IN (
'Requested', 'Accepted', 'In-Progress',
'Pending-Payment', 'Completed', 'Cancelled', 'Rejected'
)),
CONSTRAINT chk_hs_booking_rating CHECK (rating BETWEEN 1 AND 5)
);
COMMENT ON COLUMN HS_BOOKINGS.invoice_id IS
'Auto-generated by @PreUpdate hook on Completed. Format: INV<timestamp>.';

```

HS_LEAVE_REQUESTS (depends on HS_USERS)

```

CREATE TABLE HS_LEAVE_REQUESTS (
id NUMBER(19,0) NOT NULL,
provider_id NUMBER(19,0) NOT NULL,
provider_name VARCHAR2(100),
skill VARCHAR2(100),
leave_date VARCHAR2(20) NOT NULL,
time_slots VARCHAR2(500),
reason VARCHAR2(1000) NOT NULL,
status VARCHAR2(20) DEFAULT 'pending' NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
CONSTRAINT pk_hs_leave_requests PRIMARY KEY (id),
CONSTRAINT fk_hs_leave_provider FOREIGN KEY (provider_id)
REFERENCES HS_USERS (id) ON DELETE CASCADE,
CONSTRAINT chk_hs_leave_status CHECK (status IN ('pending', 'approved', 'rejected'))
);
COMMENT ON COLUMN HS_LEAVE_REQUESTS.time_slots IS
'Comma-separated time slots (e.g. 9 AM,10 AM). NULL = full-day leave.';

```

5.4 Index Creation

```

-- HS_USERS
CREATE INDEX idx_users_role ON HS_USERS (role);
CREATE INDEX idx_users_role_city ON HS_USERS (role, service_city);
CREATE INDEX idx_users_service_type ON HS_USERS (service_type);
CREATE INDEX idx_users_blocked ON HS_USERS (blocked);
CREATE INDEX idx_users_available ON HS_USERS (available);
CREATE INDEX idx_users_city ON HS_USERS (city);
CREATE INDEX idx_users_name ON HS_USERS (UPPER(name)); -- case-insensitive search
CREATE INDEX idx_users_created_at ON HS_USERS (created_at);
-- HS_BOOKINGS
CREATE INDEX idx_bookings_customer_id ON HS_BOOKINGS (customer_id);
CREATE INDEX idx_bookings_provider_id ON HS_BOOKINGS (provider_id);
CREATE INDEX idx_bookings_status ON HS_BOOKINGS (status);
CREATE INDEX idx_bookings_date ON HS_BOOKINGS (booking_date);
CREATE INDEX idx_bookings_service ON HS_BOOKINGS (service);
CREATE INDEX idx_bookings_created_at ON HS_BOOKINGS (created_at DESC);
CREATE INDEX idx_bookings_prov_date_time ON HS_BOOKINGS (provider_id, booking_date,
booking_time);
-- HS_LEAVE_REQUESTS
CREATE INDEX idx_leaves_provider_id ON HS_LEAVE_REQUESTS (provider_id);
CREATE INDEX idx_leaves_status ON HS_LEAVE_REQUESTS (status);
CREATE INDEX idx_leaves_date ON HS_LEAVE_REQUESTS (leave_date);
-- HS_PROMO_CODES
CREATE INDEX idx_promo_active ON HS_PROMO_CODES (active);
CREATE INDEX idx_promo_expires_at ON HS_PROMO_CODES (expires_at);

```

5.5 Additional Constraints (ALTER TABLE)

```

ALTER TABLE HS_BOOKINGS
ADD CONSTRAINT chk_hs_booking_amount CHECK (amount >= 0);
ALTER TABLE HS_USERS
ADD CONSTRAINT chk_hs_user_age CHECK (age > 0 AND age < 120);
ALTER TABLE HS_USERS
ADD CONSTRAINT chk_hs_reliability CHECK (reliability_score BETWEEN 0 AND 100);

```

6. Key Queries

Provider Search

```
-- Filter by city, service type, and name (GET /api/providers)
SELECT * FROM HS_USERS
WHERE role = 'provider'
AND blocked = 0
AND UPPER(service_city) LIKE UPPER('%Chennai%')
AND UPPER(service_type) LIKE UPPER('%Plumbing%')
AND UPPER(name) LIKE UPPER('%Ravi%');
```

Geolocation — Bounding Box Pre-filter

```
-- Pre-filter before Haversine calculation in the application layer
SELECT * FROM HS_USERS
WHERE role = 'provider'
AND blocked = 0
AND latitude BETWEEN :lat - :deltaLat AND :lat + :deltaLat
AND longitude BETWEEN :lon - :deltaLon AND :lon + :deltaLon;
```

Booking Queries

```
-- Customer booking history
SELECT * FROM HS_BOOKINGS
WHERE customer_id = :customerId
ORDER BY created_at DESC;
-- Provider job list
SELECT * FROM HS_BOOKINGS
WHERE provider_id = :providerId
ORDER BY created_at DESC;
-- All bookings by status (admin)
SELECT * FROM HS_BOOKINGS
WHERE status = 'Requested'
ORDER BY created_at ASC;
```

Leave Conflict Check

```
-- Before approving leave – check active booking at same date/time
SELECT COUNT(*) FROM HS_BOOKINGS
WHERE provider_id = :providerId
AND booking_date = :leaveDate
AND booking_time = :timeSlot
AND status NOT IN ('Cancelled', 'Rejected');
```

Promo Code Lookup

```
-- Validate promo code at checkout
SELECT * FROM HS_PROMO_CODES
WHERE code = :code
AND active = 1
AND (expires_at IS NULL OR expires_at > CURRENT_TIMESTAMP);
```

Analytics Queries

```
-- Total revenue
SELECT COALESCE(SUM(amount), 0) AS total_revenue
FROM HS_BOOKINGS
WHERE status = 'Completed';

-- Revenue by month
SELECT COALESCE(SUM(amount), 0) AS monthly_revenue
FROM HS_BOOKINGS
WHERE status = 'Completed'
AND booking_date LIKE '2026-05%';

-- Category demand
SELECT service, COUNT(*) AS bookings
FROM HS_BOOKINGS
GROUP BY service
ORDER BY bookings DESC;

-- Booking status distribution
SELECT status, COUNT(*) AS cnt
FROM HS_BOOKINGS
GROUP BY status
ORDER BY cnt DESC;
```

MongoDB Queries

```
// Unread notifications for a user
db.notifications.find(
{ user_id: 12, read: false },
{ sort: { created_at: -1 }, limit: 50 }
);

// Open disputes (admin dashboard)
db.disputes.find(
{ status: 'Open' },
{ sort: { created_at: -1 } }
);

// Dispute by booking ID
db.disputes.findOne({ booking_id: 45 });

// Mark notification as read
db.notifications.updateOne(
{ _id: ObjectId('...') },
{ $set: { read: true } }
);

// Append update to dispute timeline
db.disputes.updateOne(
{ _id: ObjectId('...') },
{ $push: { updates: { id: 'upd_003', actor: 'Admin', actor_role: 'admin',
note: 'Reviewed.', at: '26 May 2026, 3:00 PM' } } }
);
```